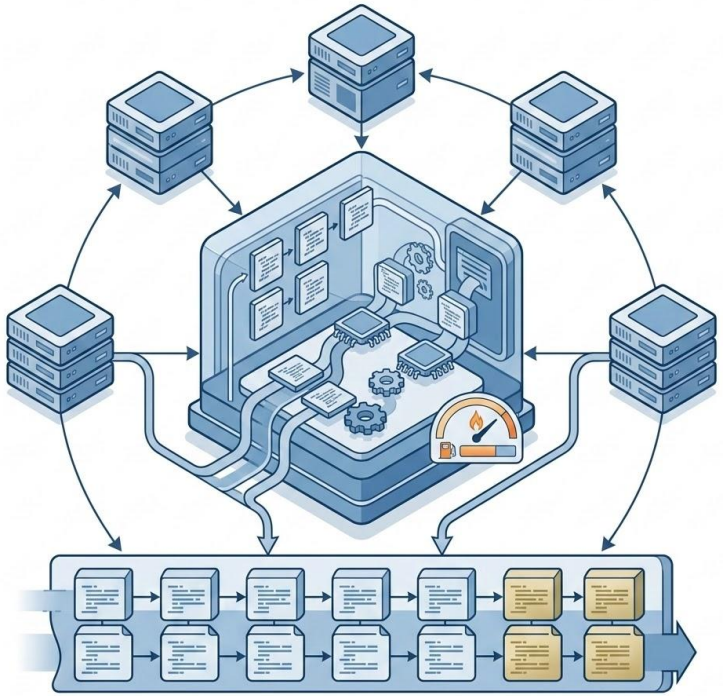
Blockchain and Cryptocurrencies Technologies

Smart Contracts in EVM

EVM and Solidity

- Smart contracts are programs deployed at blockchain addresses
- They hold state and expose functions
- They execute inside the Ethereum Virtual Machine
- They are called through transactions or read only calls
- They can hold and move assets
- They are public by default in a practical sense
- The goal today is not to become a Solidity developer
- The goal is to understand what it means to program state on a blockchain
- A smart contract is not a backend
- It is a public state machine with money attached

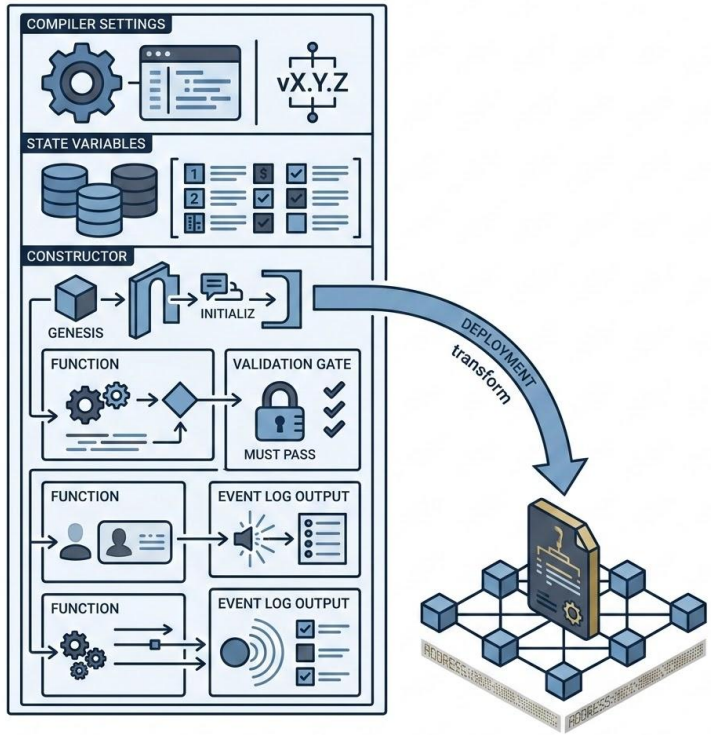
The EVM Mental Model



- The EVM executes bytecode, not Solidity directly
- Solidity is compiled into bytecode that the EVM can run
- Every node must reach the same result
- This is why execution must be deterministic
- Contract state is persistent and replicated by the network
- Transactions request changes to global state
- Read only calls inspect state without changing it
- Gas limits computation and storage abuse
- Storage is expensive because every full node must carry it
- The EVM is boring on purpose
-
- Determinism is the price of consensus

Solidity in One Slide

- pragma selects compiler compatibility
- contract defines deployable code
- State variables persist on chain
- constructor runs once during deployment
- Functions expose behavior
- view functions read state without changing it
- external functions are called from outside the contract
- msg.sender identifies the caller
- require rejects invalid execution and reverts changes
- events create logs for applications and users
- Visibility and access control are security decisions, not syntax decoration

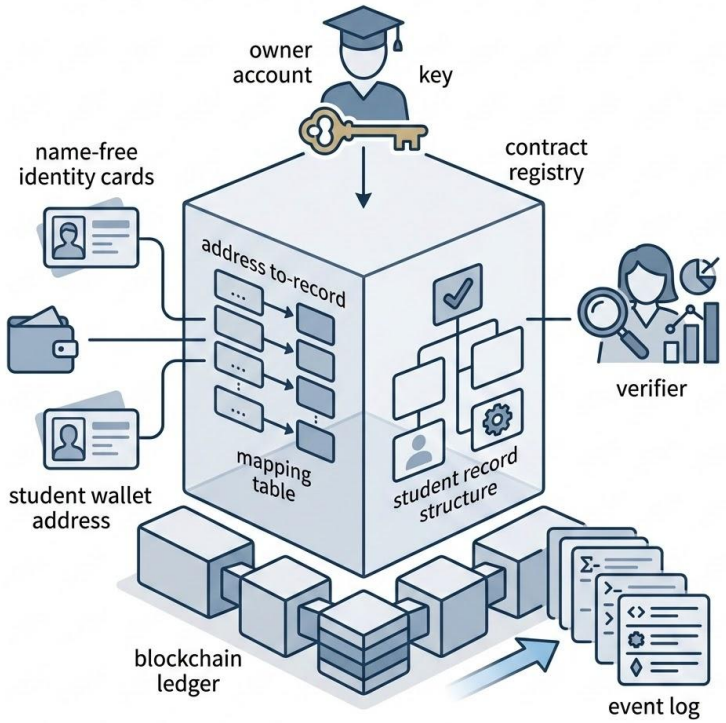


Transactions vs Read Only Calls

- A read only call inspects state locally
- It does not change blockchain state
- It does not need to be mined
- It does not consume real gas from the user
- A transaction requests a state change
- It must be signed by an account
- It is included in a block
- It consumes gas if executed on a real network
- It can fail and revert state changes
- In Remix VM, gas and accounts are simulated
- On a real network, mistakes have economic cost
- A function call in Solidity is not always the same kind of operation

Today’s Contract: CourseRegistry

- One owner controls student registration
- Students are represented by Ethereum addresses
- A mapping stores records by address
- A struct stores registration status and timestamp
- The constructor sets the deployer as owner
- registerStudent changes contract state
- isRegistered reads state
- registeredAt reads timestamp data
- require enforces access control and input validation
- events report successful registration
- This contract is simple, but it contains the core Solidity model

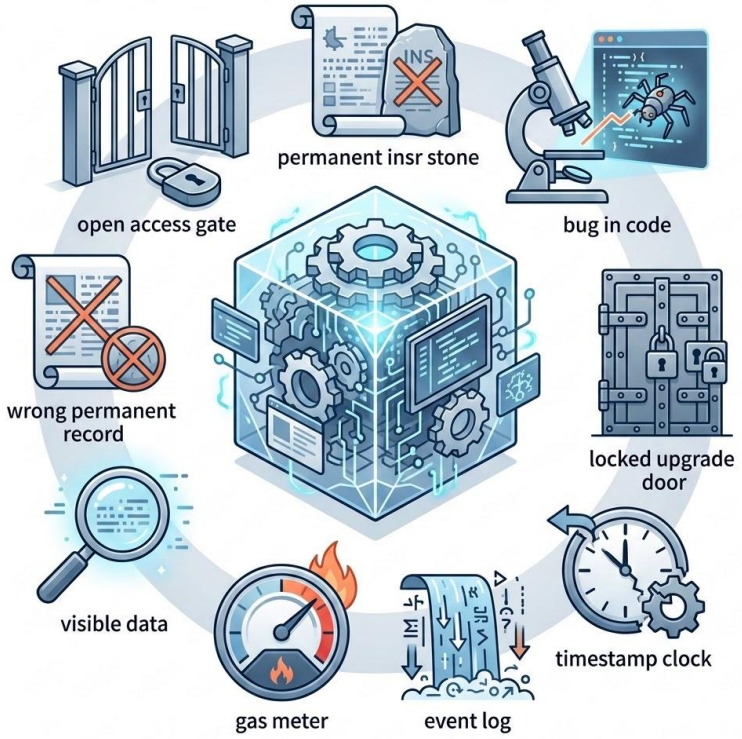


Practical Flow in Remix

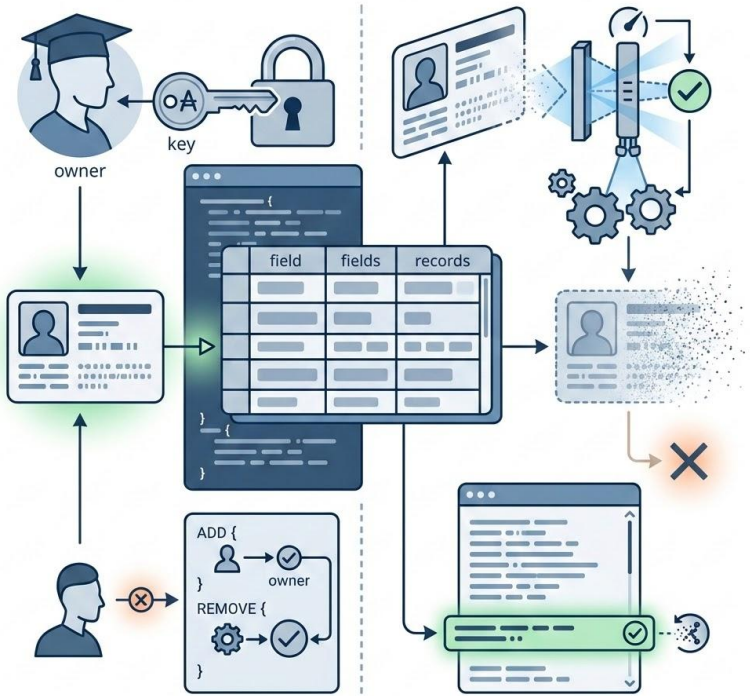
- Open Remix IDE in the browser
- Create a file named CourseRegistry.sol
- Paste the contract code
- Select Solidity compiler 0.8.x
- Compile the contract
- Deploy using Remix VM
- Check the owner address
- Register a student address
- Query isRegistered
- Query registeredAt
- Change account and test access control
- Inspect the emitted event in the transaction log
- Do not use a real wallet or real network today

What Could Go Wrong?

- Missing access control lets anyone register students
- Wrong data may become permanent
- private variables are not secret on chain
- Gas costs matter on real networks
- Events are useful, but they are not contract state
- block.timestamp should not be used for critical security decisions
- Upgradeability is not automatic
- A deployed contract cannot simply be edited like a normal backend
- Smart contract bugs can become irreversible events
- Access control is not a detail
- It is the difference between a contract and a public vandalism interface



Mini Challenge: removeStudent



- Add a removeStudent function
- Only the owner can remove students
- The student must already be registered
- Use delete records[student]
- Emit a StudentRemoved event
- Compile again
- Deploy again
- Register a student
- Remove the student
- Check isRegistered after removal
- Change account and test failed removal
- The goal is to practice state change, access control, validation, and events